

Blackjack w projekcie CasinoRoyale

Ten dokument opisuje pełne działanie modułu Blackjack w aplikacji CasinoRoyale.

Opis obejmuje backend ASP.NET Core, Entity Framework, model bazy danych, API, logikę gry, obsługę salda, split, double face up, double face down oraz warstwę frontendową.

Stan aktualny po poprawkach: gra domyka przegrane rundy, pozwala wystartować nową rozgrywkę, pokazuje aktywną rekę przy splicie, rozlicza split osobno per reka i prowadzi wolniejsze animacje.

Główne pliki

Controllers/BlackjackController.cs - API HTTP dla startu gry i akcji gracza.

Services/BlackjackService.cs - cała logika talii, rozdania, punktacji, splitu, double i rozliczenia.

Models/BlackjackGame.cs - encja zapisywana w tabeli BlackjackGames.

Models/BlackjackCard.cs - karta z kolorem, rangą i flagą FaceDown.

Data/Automaty.cs - DbContext oraz konfiguracja tabeli BlackjackGames.

wwwroot/js/games/blackjack.js - renderowanie kart, akcje przycisków, animacje i sekwencja końca rundy.

wwwroot/css/blackjack.css - wygląd stołu, kart, aktywnej reki i wyników.

Model danych BlackjackGame

Każda partia jest zapisywana jako wiersz w tabeli BlackjackGames.

DeckJson przechowuje pozostałe karty w talii po rozdaniu i kolejnych dobieraniach.

PlayerHandJson przechowuje pierwszą rekę gracza.

DealerHandJson przechowuje rekę krupiera; druga karta startowa jest ukryta przez FaceDown.

SplitHandJson przechowuje drugą rekę po wykonaniu splitu.

BetAmount to stawka głównej reki.

SplitBetAmount to stawka drugiej reki po splicie.

IsActive mówi, czy partia nadal trwa.

IsPlayerTurn mówi, czy gracz może klikać akcje.

IsSplitActive mówi, czy split został włączony.

IsPlayingSplitHand mówi, czy obecnie grana jest druga reka.

PayoutProcessed zabezpiecza przed wielokrotnym wypłaceniem tej samej rundy.

Result i WinAmount przechowują wynik całości rundy.

CreatedAt oraz FinishedAt pozwalają śledzić czas partii.

Start rundy

Frontend wysyla POST /api/blackjack/start z polem bet.

Kontroler pobiera identyfikator uzytkownika z ciasteczka logowania.

Serwis sprawdza, czy stawka jest wieksza od zera.

Jesli uzytkownik ma aktywna niedokonczona gre, zostaje ona oznaczona jako abandoned.

BalanceService.PlaceBetAsync pobiera stawke z salda.

Tworzona jest potasowana talia 52 kart.

Gracz dostaje dwie karty, krupier dostaje jedna jawna i jedna zakryta.

Stan gry jest serializowany do JSON i zapisywany w bazie.

API zwraca state oraz aktualne saldo.

Punktacja

Karty 2-10 maja wartosc zgodna z ranga.

J, Q i K maja wartosc 10.

As startowo ma wartosc 11.

Jesli suma przekracza 21, asy sa kolejno obnizane z 11 do 1.

Karty FaceDown nie licza sie do widocznej punktacji.

Blackjack to dwie karty o lacznej wartosci 21.

Bust oznacza wartosc wieksza niz 21.

Hit

POST /api/blackjack/hit dobiera jedna karte do aktualnie granej reki.

Jesli split nie jest aktywny, karta trafia do PlayerHandJson.

Jesli aktywna jest druga reka splitu, karta trafia do SplitHandJson.

Gdy pierwsza reka po splicie przekroczy 21, gra automatycznie przechodzi na druga reke.

Gdy ostatnia aktywna reka przekroczy 21, backend od razu domyka runde.

To naprawia blad, w ktorym gracz po przegranej nie mogl zaczac kolejnej gry.

Stand

POST /api/blackjack/stand konczy ture aktualnej reki.

Jesli gracz stoi na pierwszej rece po splicie, backend przelacza na druga reke.

Jesli to ostatnia reka, backend przekazuje ture krupierowi.

Krupier odsłania zakryta karte i dobiera do wartosci co najmniej 17.

Po dobraniu backend rozlicza wynik i zapisuje koniec rundy.

Double

POST /api/blackjack/double podwaja stawke glownej reki.

Double jest dostepny tylko gdy glowna reka ma dokladnie dwie karty.

Backend pobiera druga stawke z salda przez BalanceService.PlaceBetAsync.

Do reki gracza trafia dokladnie jedna karta.

Po tej karcie tura gracza sie konczy i krupier rozgrywa swoja reke.

Frontend ma dwa tryby: Face up i Face down.

Face up pokazuje karte double od razu, potem czeka i powoli przechodzi do krupiera.

Face down najpierw pokazuje karte zakryta, trzyma ja chwile, potem odsłania i dopiero po pauzie pokazuje ruch krupiera.

Sam backend rozlicza identycznie oba tryby; roznica jest prezentacyjna.

Split

POST /api/blackjack/split jest dostepny, gdy dwie startowe karty maja taka sama wartosc.

Backend pobiera druga stawke rowna BetAmount.

Pierwsza karta zostaje w glownej rece i dostaje jedna nowa karte.

Druga karta przechodzi do SplitHandJson i tez dostaje jedna nowa karte.

Frontend podswietla aktualnie grana reke etykieta w grze.

Po zakonczeniu rundy kazda reka dostaje osobny wynik.

Jesli jedna reka wygra, a druga przegra, globalny baner nie mowi juz po prostu przegrana.

Zamiast tego pokazuje Split rozliczony osobno, a przy kazdej rece widac Wygrana, Przegrana albo Remis.

Rozliczenie wygranych

Bez splitu blackjack wypłaca stake plus 1.5x stake jako zysk, czyli razem 2.5x stawki.

Zwykla wygrana wypłaca 2x stawki, czyli zwrot stawki plus zysk.

Remis wypłaca 1x stawki, czyli zwrot.

Przegrana wypłaca 0.

Przy splicie kazda reka jest rozliczana osobno funkcja ResolveHand.

WinAmount jest suma wyplat obu rak.

BalanceService.PayoutAsync dopisuje wyplate tylko raz po zakonczeniu rundy.

PayoutProcessed jest ustawiane na true, aby uniknac podwojnej wyplaty.

API

GET /api/blackjack/balance zwraca saldo uzytkownika.

POST /api/blackjack/start przyjmuje { bet }.

POST /api/blackjack/hit przyjmuje { gameId }.

POST /api/blackjack/stand przyjmuje { gameId }.

POST /api/blackjack/double przyjmuje { gameId, faceDown }.

POST /api/blackjack/split przyjmuje { gameId }.

Kazda akcja zwraca state i opcjonalnie balance.

State zawiera karty, wartosci punktowe, flagi aktywnosci, mozliwe akcje, wynik i handResults.

Frontend i animacje

blackjack.js renderuje karty jako elementy HTML z klasami bj-card, bj-card-face albo bj-card-back.

Nowe tempo jest celowo wolniejsze: akcja gracza, pauza, odsloniecie krupiera, dobieranie kart krupiera po jednej, pauza, wynik.

TIMING.doublePeek steruje czasem trzymania karty double.

TIMING.revealDealer steruje pauza po odslonieciu krupiera.

TIMING.dealerCard steruje tempem dobierania kolejnych kart krupiera.

CSS wydłuża animacje rozdania i odslonienia, aby calosc byla plynniejsza.

Przy splicie JS dodaje klasy bj-hand-active, bj-hand-win, bj-hand-lose lub bj-hand-push.

Migracje i baza

Tabela BlackjackGames jest dodana migracja AddBlackjackGames.

DbContext Automaty zawiera DbSet<BlackjackGame>.

Pola pieniezne BetAmount, SplitBetAmount i WinAmount maja decimal(18,2).

Aplikacja uruchamia dbContext.Database.Migrate() przy starcie, wiec brakujace migracje powinny wejsc automatycznie.

Do lokalnego uruchomienia baza SQL Server moze byc odpalona przez docker compose.

Najwazniejsze poprawki

1. BlackjackService jest zarejestrowany w DI, wiec kontroler API moze dzialac.
2. Dodano tabele BlackjackGames i zastosowano migracje.
3. Hit po bust domyka runde, zamiast zostawiac aktywna gre bez tury gracza.
4. Split zwraca handResults dla obu rak.

5. Frontend pokazuje wynik per reka przy splicie.
6. Double face up i face down maja plynniejsza sekwencje i wolniejszy ruch krupiera.
7. Globalny wynik splitu nie ukrywa czesciowej wygranej.